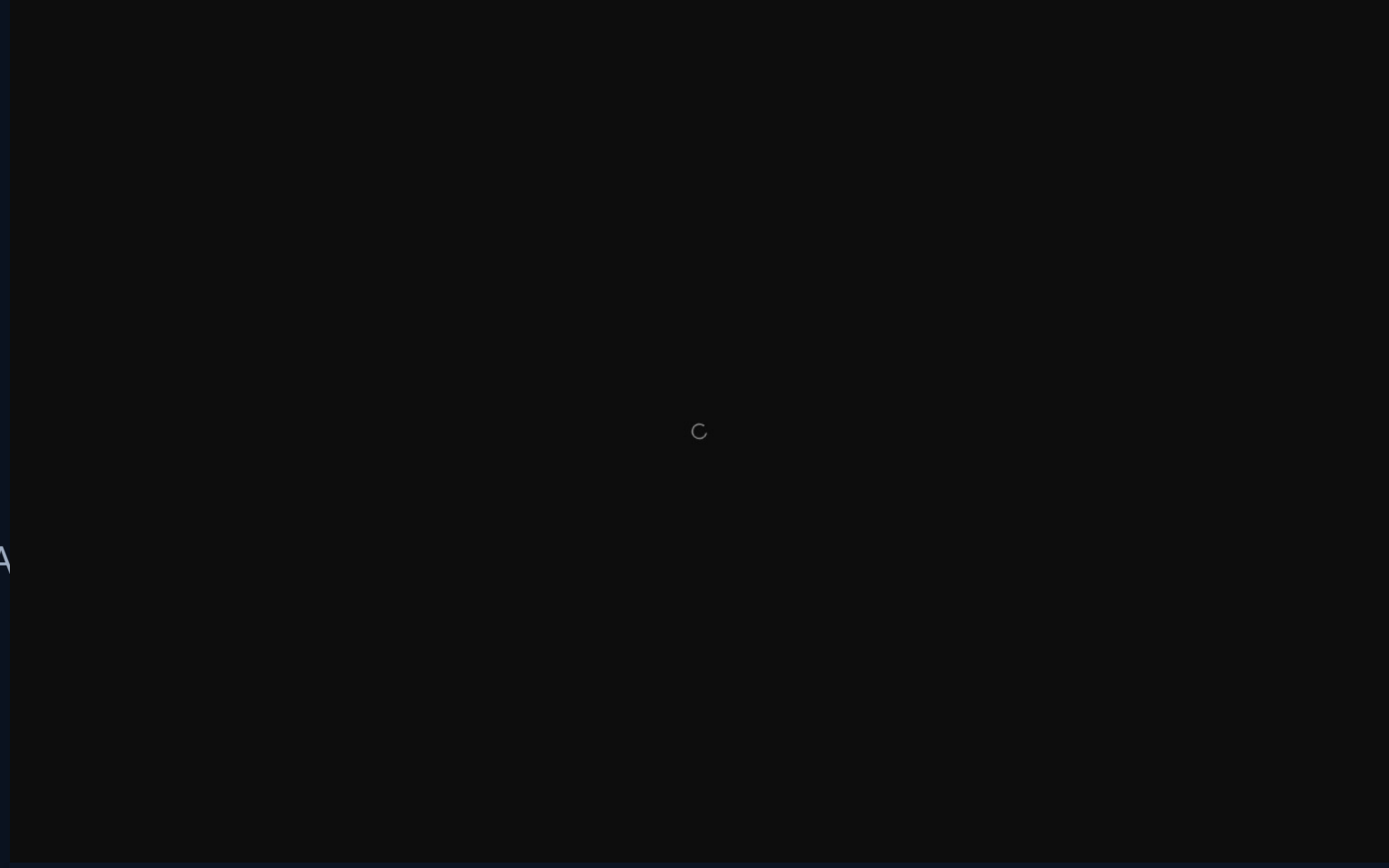
PROJECT OVERVIEW

VIGIL

Agentic Video Analytics

rsistent identity, explainable threat detection, and A

SERVICE	CONSOLE	STATUS
vigil.main	HTTPS :8090	LIVE



A private, on-premises command center

React console + FastAPI backend + GPU perception + local LLM/VLM. Data and inference remain

Technical overview · 10 slides

What VIGIL delivers

From camera pixels to reviewable, evidence-backed incidents

SEE

CCTV, RTSP, FRS relay, files, and browser webcam

UNDERSTAND

People, pose, objects, faces, motion, zones

VERIFY

Multi-stage, path-aware threat checks with evidence

ACT

Correlated incidents, investigations, reports, notes and not

Operational outcomes

Persistent IDs — recognize returning people across cameras and restarts.

Explainable alerts — retain measurements, thresholds and reasoning for audit.

Fast response — live WebSockets and evidence clips shorten review time.

Human control — watchlist matches remain candidates, not confirmed identities.

Local intelligence — Qwen text/vision models run through local Ollama.

Safe demos — synthetic data is labelled, isolated, and removable.

System architecture

One uvicorn process; camera and GPU work in owned threads; async I/O and persistence



FRS backend · :8000

MJPEG relay, watchlist and WAL code, detections, and gestures.

VIGIL data layer

SQLite in WAL mode by default; PostgreSQL optional. Clean separation between

Local AI · :11434

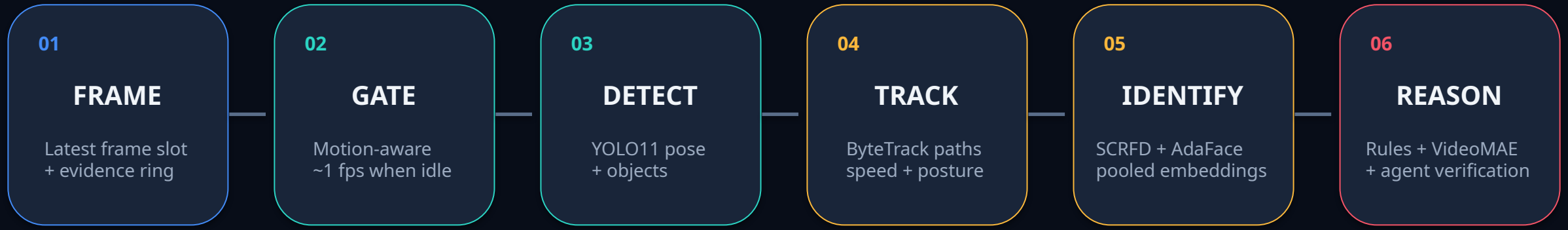
Ollama serves qwen3.5:9b for reasoning and qwen3-vl:8b for visual evidence verification. Nothing

Concurrency design: model groups own their CUDA/ONNX contexts; camera calls return Futures; bounded queues prevent slow clients from blocking perception.

04

Real-time perception pipeline

Optimized for continuous multi-camera operation and explainable decisions



Identity confidence logic

Observe — quality ≥ 0.35 and face ≥ 40 px; keep the best track views.

Match — pooled AdaFace similarity ≥ 0.40 with ≥ 0.06 runner-up margin.

Lock — two votes establish the identity; contradictory views can unlock it.

Evidence lifecycle

Capture — annotated keyframe immediately; 6 s pre-roll + 4 s post-roll.

Package — H.264 clip plus six frames for visual verification.

Preserve — rejected alerts are downgraded, not deleted, for auditability.

Analytics and detection capabilities

Deterministic perception first; AI reasoning only where it adds value

IDENTITY

Persistent person IDs
Enrollment without losing history
FRS watchlist candidates

BEHAVIOR

Loitering · running · falls
restricted zones · crowding
pacing · repeat visits

VIOLENCE

Pose interaction → VideoMAE
fusion → VLM verification
critical escalation

SEARCH

Identity timelines
co-occurrence and camera paths
CLIP appearance search

LIVE

Boxes · labels · trails
skeletons · zones · objects
backpressured WebSockets

EVIDENCE

Keyframes and clips
verification frames
incident-ready audit trail

Deliberate safeguards: clothing never assigns identity · VideoMAE never alerts alone · watchlist hits require human verification.

Agentic investigation layer

Agents enrich, verify and investigate signals while preserving every step

Threat Agent — Checks evidence frames; confirms, rejects or holds for review

Identity Agent — Adds FRS candidate context and live identity evidence

Behavior Agent — Finds repeat visits and correlated suspicious patterns

Investigation Agent — Runs a ReAct loop over ~20 read/action tools.

Reporting Agent — Produces structured findings and incident reports.

Analyst Agent — Read-only chat for people, cameras, events and incidents

Event → verified incident

SIGNAL
rule/model output

EVIDENCE
clip + frames

VERIFY
Threat Agent + VLM

CORRELATE
person · camera · type

INVESTIGATE
LLM + tools

REPORT
assessment + actions

Stored trace: thoughts · tool calls · results · decisions

Application, APIs and data

A single deployable service with an integrated browser console

React + TypeScript console

events, incidents, identities, analytics, cameras, agents

FastAPI service

REST, authentication, media

per-camera live streams, browser WebSocket

Persistence

SQLAlchemy 2.0 async with SQLite WAL; optional PostgreSQL. Additive migrations

Core records

users + audit	roles and mutations	**cameras + zones**	sources and rules	**identities + vectors**	persistent people
tracks	paths and crops	**events + incidents**	evidence and correlation	**agent_runs**	full reasoning traces

Media: data/media/evidence · faces · persons · uploads | Occupancy: data/heat/<camera>.npy

Security and integration boundaries

Designed for on-premises use with explicit trust and audit controls

Session security

JWT in HttpOnly, Secure, SameSite=Strict cookies; bearer tokens supported.

Access control

Server-side viewer < analyst < admin roles; disabled users lock immediately.

Auditability

Every mutation and every agent action is written to the audit log.

Media safety

Uploads are not served directly; paths are confined to the media root.

FRS integration contract

READ ONLY

Allowed — camera relay, watchlist search, records, detections, alerts and health.

Blocked by design — FRS enrollment, deletion, settings, re-index and alert acknowledgement.

Credential hygiene — RTSP credentials are redacted before reaching VIGIL APIs, DB or logs.

Privacy — text and vision reasoning use local Ollama; no cloud dependency is required.

Hardware profile and sizing

Current host verified locally; sizing tiers are practical deployment guidance

CURRENT HOST

NVIDIA GB10

CPU — ARM64, 20 cores (Cortex-X925/A725)

Memory — 121 GiB system/unified memory

Storage — 3.7 TB NVMe; 2.1 TB free at capture

GPU driver — NVIDIA 580.173.02

Software — CUDA 13 stack; PyTorch + ONNX Runtime GPU

Well suited to multi-model, on-prem inference

MINIMUM / DEMO	1–2 streams	8 CPU cores · 32 GB RAM NVIDIA GPU with 12+ GB VRAM 500 GB NVMe
RECOMMENDED	4–8 streams	16+ CPU cores · 64–128 GB RAM 24–48 GB VRAM / unified memory 1–2 TB NVMe
SCALE-OUT	8+ streams	Separate ingest/inference nodes PostgreSQL + shared object storage GPU capacity sized by measured FPS

Final capacity depends on resolution, analysis FPS, active motion, camera count, and concurrent LLM/VLM work.

Deployment and operational readiness

A compact, service-managed deployment with observable health and safe recovery

BUILD

Python venv + CUDA packages
Fetch YOLO / VideoMAE / CLIP
Build React frontend

CONFIGURE

.env with VIGIL_* settings
TLS certificate and keys
FRS + Ollama endpoints

RUN

systemd user service
One uvicorn worker only
HTTPS console on :8090

OBSERVE

/api/health + /api/ready
/system/status + /metrics
rotating structured logs

Recommended next steps

01 Validate camera-by-camera FPS and latency under real motion.

02 Tune thresholds and zone rules using local evaluation clips.

03 Set retention, backup and incident-response policies.

04 Load-test concurrent streams plus an active investigation.

05 Move to PostgreSQL/object storage when scaling beyond one node.

VIGIL turns continuous video into explainable, reviewable intelligence—while keeping identities, evidence and AI inference under local control.